Bayesian-Optimized Variational Quantum Circuits on a Classification Task:
A Comparison with Classical Neural Networks

Jialun Li Juan Vera

October 15, 2026

Abstract

We train a two-qubit, 24-parameter variational quantum circuit (VQC) to classify the two-moons dataset using Bayesian optimization (BO), a gradient-free method that treats the circuit loss as a black box, and compare it with classical neural networks trained by backpropagation. The circuit uses data re-uploading so that its output is a trainable Fourier series in the inputs. We find that Gaussian-process BO over all 24 parameters at once is no better than random search (79 % test accuracy), but that running BO on one circuit layer at a time recovers 93 %, and a short derivative-free local polish brings the circuit to 96.0 % ± 0.0 % over three seeds, on par with a 17-parameter multilayer perceptron (95.3 %) and a 1185-parameter one (96.0 %). The trained circuit loses under one point of accuracy when read out with 256 measurement shots. We also report that on an exact simulator the local optimizer COBYLA alone matches the BO pipeline, so the case for BO in this setting rests on the noisy-evaluation regime that our experiment does not probe. Code, results and a live terminal dashboard are provided.

1 Introduction

Variational quantum circuits are parametrized unitaries whose expectation values serve as a machine-learning model; a classical optimizer adjusts the parameters to minimize a loss [1]. On hardware, gradients are available only through the parameter-shift rule [2], which costs two extra circuit executions per parameter per data point, and every execution is a noisy estimate. This makes sample-efficient, gradient-free optimizers attractive, and Bayesian optimization [3, 4] is the canonical such method: it fits a probabilistic surrogate to all loss values seen so far and chooses each new query to maximize expected improvement.

This project asks two concrete questions on a toy task small enough to simulate exactly:

1. Can a BO-trained VQC reach the accuracy of a classical neural network with a comparable number of parameters?

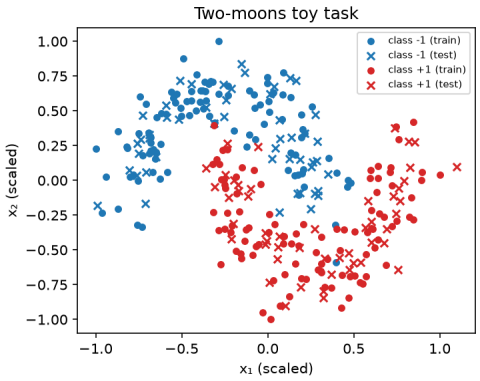


Figure 1: The two-moons task after scaling. Dots are training points, crosses test points.

2. Does BO actually help, compared with random search and with a simpler derivative-free local optimizer at the same evaluation budget?

2 Model

Task. Two interleaving moons (`sklearn.make_moons`, noise 0.2), N=300 points, split 200/100 into train/test, features scaled to $[-1, 1]^2$, labels $y \in \{-1, +1\}$ (Fig. 1). A linear classifier reaches about 84 %; the rest requires a curved boundary.

Circuit. We use a two-qubit data re-uploading circuit [5] with L=4 layers. Layer ℓ applies

$$U_\ell(\mathbf{x};\boldsymbol{\theta}) = \mathrm{CZ}\cdot\bigotimes_{q=1}^{2} R_Y(\beta_{\ell q})R_Z(\alpha_{\ell q})\cdot\bigotimes_{q=1}^{2} R_Y(w_{\ell q}\,x_q), \quad (1)$$

i.e. feature x_q is encoded on qubit q with a *trainable* scale $w_{\ell q}$, followed by a trainable rotation and a CZ entangler. Six parameters per layer give P=24 in total. The output is

$$f(\mathbf{x};\boldsymbol{\theta}) = \langle 00|\, U^\dagger (Z\otimes I)\, U\, |00\rangle \in [-1, 1], \qquad \hat{y} = \operatorname{sign} f, \quad (2)$$

and the training loss is the mean squared error $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N}\sum_i \big(f(\mathbf{x}_i;\boldsymbol{\theta}) - y_i\big)^2$.

Because $R_Y(wx) = \cos\frac{wx}{2} I - i \sin\frac{wx}{2} Y$, every matrix element of an encoding gate is a combination of $e^{\pm iwx/2}$, and f is a finite Fourier series in $\mathbf{x}$ whose frequencies are signed sums of the $w_{\ell q}$ [6]. The rotation angles set the Fourier coefficients and the trainable scales set the spectrum; each re-upload adds frequencies. A single encoding layer would give one frequency per feature, far too smooth for the moons.

Simulation. The loss is evaluated exactly with Qiskit's `StatevectorEstimator` [7], batched over the 200 training points. After training we also re-evaluate the best circuit with 256 shots per point on `AerSimulator`, where $\hat{f} = \frac{1}{S}\sum_b (-1)^{b_0} n_b$ has standard error $\leq 1/16$.

3 Optimizers

All circuit optimizers see only scalar loss values. Budgets are in loss evaluations, each of which runs 200 circuits.

Full-space BO. `gp_minimize` from scikit-optimize [8]: a Gaussian process with an ARD Matérn-5/2 kernel over $[-\pi, \pi]^{24}$, 25 Latin-hypercube initial points, then 125 points chosen by Expected Improvement,

$$\mathrm{EI}(\boldsymbol{\theta}) = (\mathcal{L}^* - \mu)\,\Phi(z) + \sigma\,\varphi(z), \quad z = \frac{\mathcal{L}^* - \mu(\boldsymbol{\theta})}{\sigma(\boldsymbol{\theta})}. \quad (3)$$

The budget is capped at 150 because GP fitting costs $O(t^3)$ in the number of observations.

Block-coordinate BO. The same optimizer applied to one layer's six parameters at a time with the other 18 frozen, 40 calls per block with the incumbent supplied as a known point, cycling through the four layers for three sweeps (481 evaluations). Updates are accepted only on improvement, so the loss is monotone. This is in the spirit of layerwise training [9] and keeps the GP in the dimensionality range where it is informative.

Block BO + polish (main pipeline). Block BO followed by 150 iterations of COBYLA [10], a derivative-free trust-region method that fits a linear model through $P{+}1$ points, started from the block-BO solution with trust radius 0.2. Total 631 evaluations.

Controls. Random search over $[-\pi, \pi]^{24}$ and COBYLA from a random start, each with 631 evaluations.

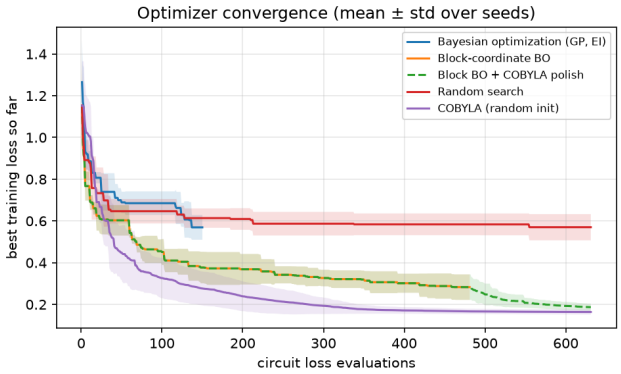


Figure 2: Best training loss versus loss evaluations, mean ± std over seeds. Full-space BO (blue) overlaps random search (red). Block BO (orange, hidden under the dashed green curve for its first 481 evaluations) descends steadily; the COBYLA polish (dashed) continues from it. COBYLA from a random start (purple) converges fastest.

Classical baselines. Logistic regression (3 parameters); a 2–4–1 multilayer perceptron with tanh units (17 parameters, the parameter-matched comparison); and a 2–32–32–1 MLP (1185 parameters). All are trained with L-BFGS on the logistic loss in scikit-learn [11].

Every method is run for three seeds, which change the data split, the BO initial design and the random starts.

4 Results

Table 1 and Figs. 2–3 summarize the experiment.

The VQC matches the classical networks. The main pipeline reaches 96.0 % test accuracy on every seed, equal to the 1185-parameter MLP and slightly above the parameter-matched MLP. With 100 test points a difference of one or two points is within noise, so the fair reading is parity. The classical models fit the training set more tightly (98.5 % to 100 % versus 96 %), so the VQC is the less expressive model and generalizes about as well.

Finite shots cost little. Reading out with 256 shots instead of exact expectation values drops test accuracy from 0.960 to 0.953 ± 0.005: the trained boundary is not relying on margins smaller than the shot-noise scale.

Full-space BO fails at P=24. Its test accuracy (0.790) is identical to random search and its convergence curve overlaps it (Fig. 2). During development, more calls (200, 300), other acquisition functions (LCB,

Table 1: Mean ± standard deviation over three seeds. Evaluations are circuit-loss evaluations (200 circuits each). The main pipeline re-evaluated with 256 measurement shots per point scores 0.953 ± 0.005 on the test set.

Model	Optimizer	Params	Train acc	Test acc	Evals
VQC	Full-space BO (GP, EI)	24	0.818 ± 0.019	0.790 ± 0.014	150
VQC	Block-coordinate BO	24	0.938 ± 0.027	0.933 ± 0.039	481
VQC	**Block BO + COBYLA polish**	24	0.960 ± 0.007	$\mathbf{0.960 \pm 0.000}$	631
VQC	Random search	24	0.812 ± 0.017	0.790 ± 0.029	631
VQC	COBYLA (random init)	24	0.972 ± 0.008	0.963 ± 0.005	631
Logistic regression	L-BFGS	3	0.855 ± 0.004	0.843 ± 0.017	–
MLP 2–4–1	L-BFGS	17	0.985 ± 0.004	0.953 ± 0.017	–
MLP 2–32–32–1	L-BFGS	1185	1.000 ± 0.000	0.960 ± 0.016	–

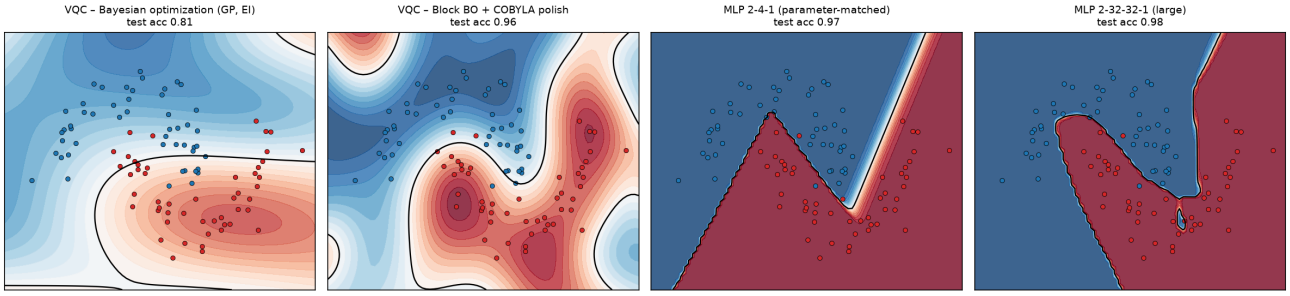


Figure 3: Decision functions on seed 0 with test points overlaid. Left to right: VQC after full-space BO; VQC after block BO + polish; parameter-matched MLP; large MLP. The black contour is the decision boundary. The VQC boundary is smooth and periodic, the MLP boundary piecewise-sigmoidal.

hedge) and larger initial designs did not change this. A GP conditioned on 150 points in a 24-dimensional periodic landscape with many local minima has a posterior close to its prior almost everywhere, so EI is nearly flat and its maximizer is effectively a random draw. This is the well-known dimensionality ceiling of GP-based BO, and VQC parameter counts cross it quickly.

Block-coordinate BO recovers most of the gap. Restricting each BO run to six parameters brings the loss from 0.57 to 0.28 and accuracy to 93 %, with the same optimizer and no gradients. It is seed-sensitive (88 % to 97 %), as coordinate descent can stop at a point that is optimal per block but not jointly; the per-sweep logs show improvements becoming rare after the second sweep.

A short local polish closes the rest. 150 COBYLA iterations from the block-BO solution remove the seed sensitivity entirely. This is the usual global-then-local pattern: BO locates a basin, a local method descends it.

The control is uncomfortable. COBYLA from a random start, with the same 631 evaluations, reaches 0.963 ± 0.005 and converges faster than the pipeline. On a noiseless simulator the landscape is smooth enough that following the local slope already works, and the global search that BO is meant to provide bought nothing here. BO's expected advantages, robustness to noisy evaluations and sample efficiency when each evaluation is very costly, are precisely the conditions this experiment does not create.

Model shapes differ. Fig. 3 shows the structural contrast between the model families. The MLP boundary is a few straight segments joined by bends, a sum of sigmoidal ridges. The VQC boundary is smooth and repeats outside the data region because f is a band-limited Fourier series in the inputs. Both fit the moons; they extrapolate very differently.

5 Discussion and limitations

Nothing here is a quantum advantage, nor could it be: a two-qubit state is a four-component vector and the whole model is a small classical linear-algebra computation. The argument for VQCs rests on regimes where the state space is too large to simulate, which a two-qubit toy cannot reach. What the experiment does establish is that a gradient-free optimizer can train such

a model to classical parity, and it characterizes which gradient-free optimizer.

The main limitation is the noiseless loss. With shot noise in the training loss, COBYLA's linear interpolation is corrupted while a GP with a noise term should remain reliable; that experiment, along with smaller budgets, is the natural next step and the one that could make BO's contribution positive rather than neutral. A second limitation is scale: at larger qubit counts barren plateaus [12] flatten the landscape and change the picture for every optimizer. Finally, the 100-point test set bounds how finely the methods can be ranked.

6 Conclusion

A 2-qubit, 24-parameter re-uploading circuit trained without gradients reaches 96 % test accuracy on two-moons, on par with classical MLPs with 17 and 1185 parameters. Gaussian-process Bayesian optimization over all parameters at once fails at this dimensionality; restricting it to one layer at a time makes it work, and a short local polish makes it reliable. On an exact simulator the simpler COBYLA optimizer is equally good, so the case for BO must be made in the noisy or very-expensive-evaluation regime.

Reproducibility. Code, per-seed results and figures are in the project repository. `run_experiment.py --seed k` reproduces one seed in about five minutes; `aggregate.py` builds the table and figures; `train_live.py` and `compare_live.py` show training live in the terminal.

References

[1] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3: 625–644, 2021. doi: 10.1038/s42254-021-00348-9.

[2] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331, 2019. doi: 10.1103/PhysRevA.99.032331.

[3] Peter I. Frazier. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.

[4] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016. doi: 10.1109/JPROC.2015.2494218.

[5] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020. doi: 10.22331/q-2020-02-06-226.

[6] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103(3):032430, 2021. doi: 10.1103/PhysRevA.103.032430.

[7] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit, 2024. arXiv:2405.08810.

[8] Tim Head, Manoj Kumar, Holger Nahrstaedt, Gilles Louppe, and Iaroslav Shcherbatyi. scikit-optimize: Sequential model-based optimization in Python, 2021. https://scikit-optimize.github.io, version 0.10.

[9] Andrea Skolik, Jarrod R. McClean, Masoud Mohseni, Patrick van der Smagt, and Martin Leib. Layerwise learning for quantum neural networks. *Quantum Machine Intelligence*, 3:5, 2021. doi: 10.1007/s42484-020-00036-4.

[10] Michael J. D. Powell. A direct search optimization method that models the objective and constraint functions by linear interpolation. In *Advances in Optimization and Numerical Analysis*, pages 51–67. Springer, 1994.

[11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[12] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018. doi: 10.1038/s41467-018-07090-4.